Android Enterprise App Management via Managed Google Play (Intune)

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Android Enterprise App Management via Managed Google Play (Intune) 5

The problem . . . 5

The solution . . . 5

What's included . . . 5

Business value . . . 5

1. Solution Overview — Android Enterprise App Management via Managed Google Play (Intune) 6

What this solution is . . . 6

App types on managed Google Play . . . 6

The four levers this solution manages . . . 6

How assignment intent maps to enrollment model . . . 7

Automation included . . . 7

Limitations (be honest with buyers) . . . 7

The standard deployment model . . . 7

2. Requirements Guide — Android Enterprise App Management via Managed Google Play (Intune) 8

2.1 A working managed Google Play connection (apps can sync) . . . 8

2.2 Enrollment model(s) decided and at least one enrolled device per model . . . 8

2.3 The app inventory and app-type decision . . . 8

2.4 Private LOB / web apps published on the Google side (if used) . . . 8

2.5 Managed configuration schema for any app you will pre-configure . . . 9

2.6 App protection (MAM) prerequisites, if you will apply APP . . . 9

2.7 Target groups for assignment (Assigned security groups) . . . 9

Checklist . . . 9

Global Requirements — Microsoft Intune Android (Android Enterprise) Solutions 10

1. Microsoft 365 tenant with Intune and Android Enterprise . . . 10

2. The managed Google Play binding (account enterprise OR Google Workspace) . . . 10

3. Administrative access (least-privilege Intune roles) . . . 11

4. Enrollment method decision and Graph/PowerShell toolchain . . . 11

5. Network reachability to Google & Microsoft endpoints . . . 11

6. A pilot scope and change control . . . 11

3. Implementation Guide — Android Enterprise App Management via Managed Google Play (Intune) 12

Step 1 — Create the assignment groups . . . 12

Step 2 — Baseline the current state . . . 12

Step 3 — Approve a store app (managed Google Play) . . . 12
Step 4 — (If used) publish and approve a private LOB or web app . . . 13
Step 5 — Assign the app to the pilot (choose intent + track) . . . 13
Step 6 — Push managed app configuration (pre-configure the app) . . . 13
Step 7 — Apply an Android app protection policy (optional but recommended) . . . 14
Step 8 — Validate the pilot (the four gates) . . . 14
Step 9 — Expand to production in waves . . . 14
5. Testing & Validation — Android Enterprise App Management via Managed Google Play (Intune) 15
Test matrix (run per enrollment model) . . . 15
Automated checks . . . 16
Evidence to capture . . . 16
Exit criteria (before Expand) . . . 16
6. Troubleshooting — Android Enterprise App Management via Managed Google Play (Intune) 16
Approved app never appears on devices . . . 17
App is Required but doesn't install silently on work-profile BYOD . . . 17
App opens blank / prompts for setup (managed config not applied) . . . 17
OEMConfig setting not taking effect (Samsung Knox / Zebra) . . . 17
App protection controls not enforced (no PIN / copy-paste still works) . . . 17
App on the wrong update track / not updating as expected . . . 18
Editing one assignment removed other groups . . . 18
Private LOB / web app not visible in Intune . . . 18
Uninstall intent didn't remove the app . . . 18
"Can't add a Google account" / enrollment-side failures . . . 18
7. Best Practices — Android Enterprise App Management via Managed Google Play (Intune) 19
Catalog governance . . . 19
Assignment discipline . . . 19
Managed configuration . . . 20
App protection (APP / MAM) . . . 20
Update-track control . . . 20
Change safety . . . 20
Hygiene . . . 20
8. Deliverables Summary — Android Enterprise App Management via Managed Google Play (Intune) 21
What you receive . . . 21
Outcomes . . . 22
Assumptions & boundaries . . . 22

Appendix A — PowerShell Scripts 23

Executive Summary — Android Enterprise App Management via Managed Google Play (Intune)

The problem

On Android Enterprise, every app is delivered through **managed Google Play** — there is no sideloading, and users see only what IT has approved. Done ad-hoc, this leaves organizations with inconsistent app catalogs, apps that arrive unconfigured (users left to set up accounts and servers by hand), no data protection on corporate apps, and uncontrolled updates that break workflows without warning. The result is support load, security gaps, and apps landing on the wrong devices.

The solution

This solution operationalizes **Android app management in Microsoft Intune** across four levers:

1. **Approve + assign** the right **app type** — store, private (line-of-business), or web apps — with the correct **intent** (Required / Available / Uninstall) per **enrollment model**.
1. **Managed app configuration** so apps arrive **pre-configured** (accounts, servers, feature flags, and OEMConfig keys for Samsung Knox / Zebra at the pattern level) — zero-touch first run.
1. **Android app protection policies** to enforce data controls (PIN, encryption, copy/paste and save-as restrictions, conditional launch) on corporate app data.
1. **Update-track control** — pilot updates on a Beta/closed track before the fleet, with a defined auto-update mode.

Delivery follows a disciplined **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** model: approve and assign to a **pilot group** first, prove four gates (Delivery, Configuration, Protection, Update track) on a device per enrollment model, then expand in waves — each with a documented rollback.

What's included

Full documentation (overview, requirements, step-by-step implementation, testing matrix, troubleshooting, best practices) plus **five PowerShell (Microsoft Graph) tools** that inventory the app catalog and assignment posture, stage assignments + managed configuration, stage app protection, validate delivery/config, and check managed Google Play connection health. Every tool is **read-only or `-WhatIf` by default**, uses least-privilege Graph scopes, and stores **no credentials**.

Business value

- **Users equipped, IT in control** — the right apps, pre-configured, protected, and update-controlled.
- **Fewer tickets** — pre-configuration eliminates manual app setup.
- **Lower risk** — corporate data protected; updates piloted, not sprung on the fleet; correct targeting.
- **Repeatable and reversible** — a governed catalog and a pilot-first, gated process with rollback.

> ***Note on evolving areas:*** *the Android Management API (AMAPI) transition, web/account-driven enrollment, and whether a change triggers device* ***re-enrollment*** *are actively evolving. This solution flags those points and directs verification against* ***current Microsoft guidance / your tenant*** *rather than asserting behavior that may have changed.*

1. Solution Overview — Android Enterprise App Management via Managed Google Play (Intune)

What this solution is

On **Android Enterprise**, all apps are delivered through **managed Google Play** — there is no sideloading in the managed profile, and the Google Play Store app in a work profile / fully managed device shows only the apps IT has **approved**. This solution covers doing that well in **Microsoft Intune**: choosing the right **app type**, applying **managed configuration**, adding **app protection**, controlling the **update track**, and **assigning** each app to the right devices by enrollment model and group.

App types on managed Google Play

- **Store apps (managed Google Play store apps)** — public apps (e.g. Outlook, Chrome) approved from

the managed Google Play iframe inside Intune and synced back as `androidManagedStoreApp` objects.

- **Private / line-of-business (LOB) apps** — your own `.apk` published privately to your organization

through the managed Google Play **Publishing / Custom app** flow, so only your tenant sees them.

- **Web apps (web links)** — a URL packaged by managed Google Play into an installable web-app icon.

- **OEMConfig apps** — a vendor's OEMConfig app (Samsung Knox Service Plugin, Zebra OEMConfig, etc.)

is a *store app* that carries a **managed configuration schema** to drive OEM-specific settings (integrated at the **pattern level** — the schema and keys come from the OEM's documentation).

The four levers this solution manages

1. **Approval + assignment** — approve the app in managed Google Play; assign it in Intune with an

intent (**Required**, **Available for enrolled devices**, **Available with/without enrollment**, or **Uninstall**) to a **group**, scoped by **enrollment model**.

1. **Managed app configuration (app config policies)** — push app settings/keys (managed

configurations) to the app so it arrives pre-configured (accounts, servers, feature flags, OEM keys) with no user setup.

1. **App protection policies (APP / MAM)** — data-protection controls (encryption, copy/paste and

save-as restrictions, PIN, conditional launch) applied to the app's corporate data. On enrolled devices these complement device policy; on unenrolled devices they are the primary control (MAM-only).

1. **Update-track control** — choose which **release track** (Production, Beta, or a custom closed

track you publish to) and **auto-update mode** an app follows via its assignment settings.

How assignment intent maps to enrollment model

• **Work profile (BYOD)** — apps land in the **work profile**; **Available** intents let users pull apps from the work Play Store; **app protection** matters most here (personal side is untouched).

• **Fully managed / corporate-owned work profile** — **Required** apps are pushed silently; the whole device (or work profile) is corporate.

• **Dedicated / COSU (kiosk)** — apps are **Required** and surfaced through the kiosk/managed home screen; user-initiated installs are typically disabled.

> ***Managed configuration and update tracks are set in the app's *assignment*.*** *The same app can be assigned to different groups with* ***different managed configs and different tracks*** *— that is how you pilot a Beta track to a test group while Production stays on the stable track.*

Automation included

Five **PowerShell (Microsoft Graph)** tools — **read-only or** `-WhatIf` **by default** — inventory the managed Google Play apps and their assignment posture, stage app assignment + managed configuration, stage an Android app protection policy, validate that delivery/config is in place, and check the managed Google Play binding and scope health. No credentials or secrets are stored in the scripts.

Limitations (be honest with buyers)

• Managed Google Play publishing (private LOB apps, custom web apps, closed test tracks) happens in the **managed Google Play / Google Play Console** surfaces — Intune approves and assigns, but the app is published on the Google side first. Those consoles evolve; steps are flagged where they may drift.

• The **assignment** `assign` **action replaces *all* of an app's assignments** — partial edits must re-send the full set. The scripts read current state and gate writes behind `-Commit` to prevent accidental clobbering.

• Whether adding an account, changing the connector, or the **AMAPI** transition triggers **re-enrollment** is version-dependent — **verify against current Microsoft guidance / your tenant**.

• OEM (Samsung Knox, Zebra) and third-party app specifics are handled at the **pattern level**; the authoritative schema/keys live in the vendor's documentation.

The standard deployment model

> ***Assess*** → ***Design*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand*** *— approve and assign to a* ***pilot group*** *first, prove delivery + config + protection + track, then expand in waves with pass/fail gates and a documented rollback (unassign / revert config) for every production change.*

2. Requirements Guide — Android Enterprise App Management via Managed Google Play (Intune)

Verify **every** requirement before you approve and assign apps. Each entry: **Description · Why · Verify · Implement-if-missing.** The six category-wide prerequisites (tenant + Intune, the managed Google Play binding, least-privilege roles, enrollment-method decision + Graph toolchain, network reachability, pilot scope + change control) are defined once in `global-requirements-android.md` — confirm those first; the items below are **specific to app management** on managed Google Play.

2.1 A working managed Google Play connection (apps can sync)

Description: The managed Google Play binding is not just present but **healthy** — Intune can sync the approved-app catalog from Google. **Why:** App approval, assignment, and managed configuration all depend on the connection; a stale or errored binding is the most common reason an approved app never appears on devices. **Verify:** Intune admin center → **Devices > Android > Android enterprise** — confirm the connection is connected and app sync is current. Run `ANDROIDAPP-Scope.ps1` (reads the managed store account enterprise settings, including last app-sync status/time). **Implement:** Repair/re-establish the binding (that is the *connection setup* solution) before app work.

2.2 Enrollment model(s) decided and at least one enrolled device per model

Description: The target enrollment model(s) — **work profile (BYOD)**, **fully managed**, **corporate-owned work profile**, or **dedicated/COSU** — chosen, with a test device enrolled in each model you will assign to. **Why:** Assignment intent and behavior differ by model (silent **Required** push vs user-pulled **Available**; kiosk surfaces apps differently). You cannot validate delivery without an enrolled device. **Verify:** Intune → **Devices > All devices** (filter Android); confirm ownership/enrollment profile. **Implement:** Enroll a device per model into the **pilot group** before assigning.

2.3 The app inventory and app-type decision

Description: A list of the apps to deliver, and for each the **type** (store / private LOB / web / OEMConfig) and its **exact managed Google Play identity** — the **package name** (e.g. `com.microsoft.office.outlook`) and **publisher**. **Why:** Managed Google Play search returns near-identical results; the package name + publisher is what guarantees you approve and lock to the *right* app, and the package name is the key the scripts and app config policies match on. **Verify:** Confirm each app's package name (managed Google Play iframe → app detail, or the vendor's docs). Run `ANDROIDAPP-Discovery.ps1` to list what is already approved. **Implement:** Record app + type + package name + intended intent per group before approving.

2.4 Private LOB / web apps published on the Google side (if used)

Description: For **private (LOB) apps**, the `.apk` published privately to your organization via the managed Google Play **Publishing** flow (Google Play Console developer account or the in-iframe publisher); for **web apps**, the web link created in managed Google Play. **Why:** Intune can only approve/assign what managed Google Play already hosts for your organization; private publishing and web-app creation happen on the Google side first. **Verify:** The app/web link appears in the managed Google Play iframe under your

organization's apps. **Implement:** Publish the private app / create the web app in managed Google Play (flagged: the Publishing surface evolves — follow current Google guidance), then sync into Intune.

2.5 Managed configuration schema for any app you will pre-configure

Description: For each app you will push settings to, the app's **managed configuration keys** (from managed Google Play's app-config UI) or, for OEMConfig, the **OEM's configuration schema**. **Why:** App config policies write **managed configurations** the app publishes; you can only set keys the app (or OEM) actually exposes. **Verify:** In the Intune app-config policy editor, the **configuration designer** lists the app's keys; for OEMConfig, the OEM app's schema is shown. Confirm the keys you need exist. **Implement:** Obtain the key list/schema from the vendor/OEM documentation before authoring the config.

2.6 App protection (MAM) prerequisites, if you will apply APP

Description: If applying **Android app protection policies**, the licensing for Intune app protection and, for **MAM-only (unenrolled)** scenarios, the **Microsoft Entra Company Portal** presence the APP framework requires on the device. **Why:** App protection policy enforcement relies on the Intune APP SDK/Company Portal broker; without it the policy does not attach to the app's corporate data. **Verify:** Intune → **Apps > App protection policies** is available; run `ANDROIDAPP-AppProtection.ps1` (read mode) to list existing Android APP policies. **Implement:** Ensure Intune app-protection licensing and Company Portal availability before assigning APP.

2.7 Target groups for assignment (Assigned security groups)

Description: Entra security groups for each assignment scope — a **pilot** group and the per-model / per-audience production groups. **Why:** Apps, configs, tracks, and protection are targeted to **groups**; clean group design is what lets you pilot a Beta track to one group while Production stays stable. **Verify:** Entra → **Groups**; confirm membership. Run `ANDROIDAPP-Discovery.ps1` to see current app→group assignments. **Implement:** Create `ANDROIDAPP-Pilot` and the production groups before assigning.

Checklist

- Category global requirements confirmed (`global-requirements-android.md`)
- Managed Google Play connection healthy / app sync current — `ANDROIDAPP-Scope.ps1`
- Enrollment model(s) decided; a test device enrolled per model in the pilot group
- App inventory with type + package name + intended intent — `ANDROIDAPP-Discovery.ps1`
- Private LOB / web apps published on the Google side (if used)
- Managed configuration keys / OEM schema obtained (if pre-configuring)
- App protection prerequisites met (if applying APP) — `ANDROIDAPP-AppProtection.ps1`
- Pilot + production Entra groups created
- Change window / rollback agreed

Global Requirements — Microsoft Intune Android (Android Enterprise) Solutions

These requirements apply to **every** solution in the **Android** category (Android Enterprise connection & managed Google Play, Google Workspace ↔ Intune EMM integration, the Android Management API (AMAPI) migration, dedicated/COSU & work-profile enrollment, app management, and enrollment troubleshooting). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Android solution).*** *The **Microsoft Intune admin center** (intune.microsoft.com), **Microsoft Graph**, the **managed Google Play** enrollment surface, and the **Google Admin console** (for Google Workspace / Cloud Identity integration) are authored at exact, current click-by-click / cmdlet detail from Microsoft's and Google's public documentation. The **Android Management API (AMAPI)** transition (Intune's move off the legacy Play EMM API) and **web-based / account-driven enrollment** are **actively evolving** — steps, defaults, and whether a change triggers **re-enrollment** are **cited or flagged "verify against current Microsoft guidance / your tenant"**, never asserted from memory or invented. **OEM specifics** (Samsung Knox, Zebra, and other rugged/kiosk vendors) and **third-party apps** are integrated at the pattern level; their consoles are deferred to the vendor's docs. Automation is **PowerShell** (Microsoft Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with Intune and Android Enterprise

• **Description:** A Microsoft 365 tenant with **Microsoft Intune** and Android Enterprise available, and the licensing the capability requires.

• **Why required:** All Android policy, enrollment, and app configuration is performed in Intune.

• **How to verify:** Sign in to the Intune admin center; confirm Android platform availability.

• **How to implement if missing:** Provision/assign Intune licensing before starting.

2. The managed Google Play binding (account enterprise OR Google Workspace)

• **Description:** Intune connected to **managed Google Play** — either the **managed Google Play account enterprise **(for tenants WITHOUT Google Workspace) or the** Google Workspace / Cloud Identity ↔ Intune third-party EMM** integration (for tenants that use Google Workspace). One and only one model applies per tenant.

• **Why required:** Android Enterprise enrollment and app delivery depend on the managed Google Play connection; the two binding models are mutually exclusive and must be chosen deliberately.

• **How to verify:** Intune admin center → **Devices > Android > Android enterprise (or Enrollment)** — confirm the connection status and which model is in place.

• **How to implement if missing:** Establish the correct binding (this is often the first solution in the line) before enrolling devices.

3. Administrative access (least-privilege Intune roles)

• **Description:** Access with the least-privilege role the solution names — typically **Intune

Administrator**, with** Global Administrator** only where genuinely required (e.g. the initial managed Google Play / Google Workspace binding).

• **Why required:** Enrollment, connector, and app changes require the corresponding role.

• **How to verify:** Confirm the role in the M365 admin center / Entra roles.

• **How to implement if missing:** Have a Global Admin assign the least-privilege role under change control.

4. Enrollment method decision and Graph/PowerShell toolchain

• **Description:** A decided **enrollment method** — personally-owned **work profile** (BYOD), **fully

managed**,** corporate-owned work profile**, or** dedicated / COSU (kiosk) **— plus a workstation with** Microsoft Graph PowerShell** for automation.

• **Why required:** Profiles, restrictions, and app assignment differ by enrollment method; the provided

scripts run through Graph.

• **How to verify:** Confirm the target enrollment method(s) with the device owners; `Connect-MgGraph`

succeeds.

• **How to implement if missing:** Choose the enrollment method and install Graph PowerShell before starting.

5. Network reachability to Google & Microsoft endpoints

• **Description:** Devices and the enrollment path can reach the required **Google (managed Google Play,

Android) **and** Microsoft (Intune, Entra)** service endpoints (no proxy/filter blocks the enrollment or account sign-in).

• **Why required:** Enrollment, managed Google Play account provisioning, and adding a Google account all

require reachability; blocked endpoints are a common cause of "can't add a Google account" failures.

• **How to verify:** Confirm the required endpoints are allowed on the device network.

• **How to implement if missing:** Allow the documented Google/Microsoft endpoints before enrolling.

6. A pilot scope and change control

• **Description:** A **pilot group / small device set** to validate on before fleet-wide rollout, and a

change-control process for production changes (enrollment-model, connector, and AMAPI-migration changes are high-impact and may affect existing devices).

• **Why required:** Every solution follows an **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**

model; the pilot proves the design and protects the production fleet.

• **How to verify:** Confirm a named pilot scope and an approval/change window are available.

• **How to implement if missing:** Create a pilot scope and agree change control before production changes.

*The deployment model across every Android solution is the same: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand**, with explicit pass/fail gates and a documented rollback for production changes. Automation is delivered as **PowerShell** (Microsoft Graph) — never with hardcoded credentials or secrets. Where the AMAPI transition or enrollment behavior is version-dependent, verify against current Microsoft guidance rather than assuming.*

3. Implementation Guide — Android Enterprise App Management via Managed Google Play (Intune)

Follow these steps **in order**, in the **Microsoft Intune admin center** (`https://intune.microsoft.com`) and the **managed Google Play** iframe surfaced inside it. Names in `code` are the exact objects to create — keep them consistent so the scripts and reports find them.

> ***Model:** approve → configure → protect → set track → **assign to** `ANDROIDAPP-Pilot` **first** → validate the four gates → expand in waves. Every production change has a rollback.*

Step 1 — Create the assignment groups

1. Entra → **Groups** → **New group** → Security → `ANDROIDAPP-Pilot` (**Assigned**); add one enrolled test

device per enrollment model you will target. **Create**.

1. Create the production groups you need (e.g. `ANDROIDAPP-Prod-WorkProfile`,

`ANDROIDAPP-Prod-FullyManaged`, `ANDROIDAPP-Prod-Dedicated`). Keep them **Assigned** for predictable app targeting.

Step 2 — Baseline the current state

1. Run `ANDROIDAPP-Scope.ps1` — confirms the managed Google Play binding is connected and app sync is

current (pass gate before you approve anything).

1. Run `ANDROIDAPP-Discovery.ps1` — records which managed Google Play apps are already approved and

how they are assigned, so you know your starting posture.

Step 3 — Approve a store app (managed Google Play)

1. Intune → **Apps > All apps > Add** → App type **Managed Google Play app** → **Select** (opens the

managed Google Play iframe).

1. Search the app by its **exact package name / publisher** (from Requirement 2.3) — e.g. **Microsoft Outlook**, publisher** Microsoft Corporation**, package `com.microsoft.office.outlook`. Disambiguate from similarly named apps.

1. **Approve** → on **Approval settings**, choose how app **permission changes** are handled. A common safe choice is **"Keep approved when app requests new permissions"** so an app doesn't silently un-approve on an update (confirm the current option wording in the iframe). **Done** → **Sync**.

1. Back in Intune, wait for **Sync** to pull the approved app into **Apps > Android** (this is the `androidManagedStoreApp` object the scripts report on).

Step 4 — (If used) publish and approve a private LOB or web app

1. **Private LOB app:** in the managed Google Play iframe → **Publish / Create a private app** (or Google Play Console developer account) → upload the `.apk`, name it, publish **privately** to your organization. **Approve** → **Sync**. *(The private-publishing surface evolves — follow current Google guidance; verify against your organization's publisher account.)*

1. **Web app:** iframe → **Web apps** → **Add** → enter title, URL, display mode, icon → **Create** → **Approve** → **Sync**. It appears in Intune as a managed Google Play (web) app you assign like any other.

Step 5 — Assign the app to the pilot (choose intent + track)

1. Intune → **Apps > Android > (the app) > Properties > Assignments > Edit** (or **Add group** on the assignment blade).

1. Add `ANDROIDAPP-Pilot` under the **intent** that matches the model:

• **Required** — silently installed (fully managed / corporate-owned work profile / dedicated).

• **Available for enrolled devices** — user pulls it from the work Play Store (work-profile BYOD).

• **Available with/without enrollment** — MAM-style availability (verify applicability to your model).

• **Uninstall** — removes the app.

1. On the assignment's **settings** (managed Google Play app settings), set the **App update priority / auto-update mode **and, if you publish to a non-Production track, select the** track **(Production / Beta / your closed track).** Review + save**.

1. Prefer to stage this with `ANDROIDAPP-AssignConfig.ps1 -WhatIf` first — it prints the exact assignment (intent, group, track, auto-update mode) it *would* apply. **Note:** the Graph `assign` action **replaces all** of an app's assignments, so the helper reads the current set and only writes (behind `-Commit`) the full intended set — preventing accidental removal of other groups.

Step 6 — Push managed app configuration (pre-configure the app)

1. Intune → **Apps > App configuration policies > Add > Managed devices**.

2. **Platform:** Android Enterprise. **Profile type:** the enrollment-model profile that applies

(e.g. **Fully Managed, Dedicated, and Corporate-Owned Work Profile Only** vs **Personally-Owned Work Profile Only**). **Associated app:** select the app from Step 3/4.

1. On **Settings**, use the **configuration designer** to set the app's **managed configuration keys**

(accounts, server URLs, feature flags). For **OEMConfig**, select the OEM app and set its schema keys (Samsung KSP / Zebra OEMConfig — keys per the OEM's documentation).

1. **Assignments** → `ANDROIDAPP-Pilot` → **Create**. The app now arrives pre-configured.

2. Stage/verify with `ANDROIDAPP-AssignConfig.ps1 -WhatIf` (it also reports whether an app-config

policy is attached for the app's package).

Step 7 — Apply an Android app protection policy (optional but recommended)

1. Intune → **Apps > App protection policies > Create policy > Android**.

2. **Public apps / Custom apps:** add the target app(s) by package name. Set **Data protection**

(backup, transfer, copy/paste, save-as), **Access requirements** (PIN, credentials), and **Conditional launch** (min OS/patch, jailbreak/root block, offline grace) to your standard.

1. **Assignments** → `ANDROIDAPP-Pilot`. **Create**.

2. Stage/verify with `ANDROIDAPP-AppProtection.ps1 -WhatIf` (read mode lists existing Android APP

policies and their assignments; `-Commit` creates the staged policy).

Step 8 — Validate the pilot (the four gates)

Run `ANDROIDAPP-Validation.ps1` and confirm on a pilot device per model:

- **G1 Delivery** — Required apps install silently; Available apps appear in the work Play Store.
- **G2 Configuration** — the app opens **pre-configured** (accounts/URLs populated); OEM keys applied.
- **G3 Protection** — APP controls are in force (PIN prompt, copy/paste/save-as blocked as configured).
- **G4 Update track** — the app is on the **intended track** and honoring the auto-update mode.

Pass = all four gates green on every targeted model. **Fail** = fix before expanding (see §6).

Step 9 — Expand to production in waves

1. Add the production groups (`ANDROIDAPP-Prod-*`) to each app's assignment under the matching intent,

with the **Production** track for the general fleet (keep Beta on a smaller group if piloting updates).

1. Roll out **wave by wave**; re-run `ANDROIDAPP-Validation.ps1` after each wave.

2. Keep new apps flowing through the same approve → config → protect → assign pattern.

> ***Rollback (any step): unassign*** *the app / config / protection from the affected group in Intune (or set the app's intent to* ***Uninstall*** *for a Required app), and* ***revert the track*** *to the prior stable track.* `ANDROIDAPP-AssignConfig.ps1 -WhatIf` *shows the current set before you change it, so you can restore it exactly. Connector/AMAPI-level changes are* ***high-impact*** *— treat separately under change control and verify re-enrollment behavior against current Microsoft guidance.*

Order of operations

`Groups → baseline (Scope/Discovery) → approve store app → (private/web) → assign to pilot (intent+track) → managed config → app protection → validate 4 gates → expand in waves`

5. Testing & Validation — Android Enterprise App Management via Managed Google Play (Intune)

Validate on the **pilot** first, one enrolled device **per enrollment model** you target. Each test has a **pass/fail** signal. Do not expand until all four gates pass on every targeted model.

Test matrix (run per enrollment model)

#	Gate	What you do	Pass signal	Fail signal
1	**G1 Delivery — Required**	Assign an app as **Required** to the pilot group; sync the device	App installs **silently**, appears in the work profile / on the managed home screen	App never installs; stuck "pending"
2	**G1 Delivery — Available**	Assign an app as **Available for enrolled devices** (work-profile BYOD)	App is **listed in the work Play Store** and installs on tap	App not offered in the work Play Store
3	**G2 Managed config**	Attach an app-config policy with known keys (e.g. a server URL / account)	App opens **pre-configured**; the pushed values are present	App opens blank / prompts for setup
4	**G2 OEMConfig** (if used)	Push an OEM key via the OEM's OEMConfig app	The OEM setting is applied on the device	OEM setting unchanged
5	**G3 App protection**	Apply an Android APP policy; open the app's corporate data	PIN prompt appears; copy/paste / save-as blocked per policy	Data controls not enforced
6	**G4 Update track**	Assign the app on the **intended track** with a chosen auto-update mode	Installed build matches the track; updates follow the mode	App on the wrong track / not updating as set

#	Gate	What you do	Pass signal	Fail signal
7	**Scope / uninstall**	Set a Required app's intent to **Uninstall** for a test group	App is removed from the device	App remains

Automated checks

• `ANDROIDAPP-Scope.ps1` — managed Google Play binding connected + **app sync current** (pre-flight).

• `ANDROIDAPP-Discovery.ps1` — inventories approved managed Google Play apps and their

assignment posture (intent, group, track, auto-update mode) so you can confirm what is targeted.

• `ANDROIDAPP-AssignConfig.ps1 -WhatIf` — prints the exact assignment + whether an app-config policy

is attached for the app's package, **without writing**.

• `ANDROIDAPP-AppProtection.ps1` (read mode) — lists Android app protection policies + assignments.

• `ANDROIDAPP-Validation.ps1` — end-to-end report: per app, does it have an assignment, a managed

config, an app protection policy, and a defined track/auto-update mode? Flags gaps.

Evidence to capture

• Screenshots: app installed (work profile / managed home screen), app **pre-configured**, PIN prompt.

• `ANDROIDAPP-Discovery.ps1` / `ANDROIDAPP-Validation.ps1` output (CSV) showing intent, track, config, APP.

• The managed Google Play **approval settings** chosen for each app (permission-change handling).

Exit criteria (before Expand)

• G1 Delivery — Required installs silently **and** Available appears, on every targeted model

• G2 Configuration — app arrives pre-configured (and OEM keys applied, if used)

• G3 Protection — APP controls enforced as configured (if APP in scope)

• G4 Update track — apps on the intended track, honoring auto-update mode

• Uninstall intent verified removes an app

• `ANDROIDAPP-Validation.ps1` shows no unexpected gaps for the pilot apps

6. Troubleshooting — Android Enterprise App Management via Managed Google Play (Intune)

Symptom → likely root cause → fix. Start with `ANDROIDAPP-Scope.ps1` (binding + app-sync health) and `ANDROIDAPP-Discovery.ps1` (assignment posture) — most app-delivery failures are a stale sync, a missing assignment, or the wrong intent for the enrollment model.

Approved app never appears on devices

- **Root cause:** managed Google Play **app sync stale/errored**, or the app isn't assigned (or assigned to a group the device isn't in), or the intent doesn't match the model.
- **Fix:** run `ANDROIDAPP-Scope.ps1` — if last app-sync isn't recent/`success`, trigger a **Sync** in **Devices > Android > Android enterprise**. Confirm the assignment with `ANDROIDAPP-Discovery.ps1`; verify the device is in the target group and the intent is **Required** (silent) for fully managed/dedicated.

App is Required but doesn't install silently on work-profile BYOD

- **Root cause:** on **work profile (BYOD)**, apps land in the work profile and users often **pull** them; a Required push may still need the work Play Store to process it, and personal-side apps are untouched.
- **Fix:** confirm the device is truly work-profile enrolled; allow time/sync. If the intent should be user-initiated, use **Available for enrolled devices** instead. Verify against current Microsoft guidance for the exact per-model behavior.

App opens blank / prompts for setup (managed config not applied)

- **Root cause:** no **app-config policy** for the app's package, wrong **profile applicability** (personally-owned work profile vs fully managed/dedicated), or the policy isn't assigned to the device's group.
- **Fix:** `ANDROIDAPP-AssignConfig.ps1 -WhatIf` reports whether a config is attached for the package.

Create/attach an app-config policy with the **matching profile type**, set the app's **managed configuration keys**, and assign it to the same group as the app.

OEMConfig setting not taking effect (Samsung Knox / Zebra)

- **Root cause:** the OEMConfig **app isn't deployed**, the wrong **schema key** was set, or the OEM requires a specific enrollment model.
- **Fix:** ensure the OEM's OEMConfig app (e.g. Knox Service Plugin, Zebra OEMConfig) is deployed and assigned, then set keys **from the OEM's documented schema** (vendor-authoritative). Confirm the device model/OS supports the setting per the OEM.

App protection controls not enforced (no PIN / copy-paste still works)

- **Root cause:** APP policy not assigned to the user/group, the app not added to the policy, or (MAM-only)

the **Company Portal**/APP broker prerequisite missing on the device.

- **Fix:** `ANDROIDAPP-AppProtection.ps1` (read mode) lists policies + assignments — confirm the app is

in the policy and the policy is assigned. Ensure Company Portal is present for MAM-only. Re-open the app's corporate data to trigger enforcement.

App on the wrong update track / not updating as expected

- **Root cause:** the assignment's **track** isn't set to the intended track, the device isn't in the

track's group, or the **auto-update mode** differs from expectation.

- **Fix:** `ANDROIDAPP-Discovery.ps1` shows Tracks + AutoUpdate per assignment. Set the intended

track and **auto-update mode** in the app's assignment settings (`ANDROIDAPP-AssignConfig.ps1 -WhatIf` to preview). For a **closed/Beta track**, the device/account must be a member of that track in managed Google Play.

Editing one assignment removed other groups

- **Root cause:** the Graph `assign` **action replaces ALL** of an app's assignments — sending a partial

set drops the rest.

- **Fix:** always read the current set first (`ANDROIDAPP-Discovery.ps1`) and re-send the **full**

intended set. `ANDROIDAPP-AssignConfig.ps1` does this automatically (preserves other groups, gated behind `-Commit`). Restore a dropped group by re-adding it in the full set.

Private LOB / web app not visible in Intune

- **Root cause:** the app was **published on the Google side** but not yet synced/approved, or published

to the wrong organization.

- **Fix:** confirm it appears in the managed Google Play iframe under your organization, **Approve**, and

Sync. The private-publishing surface evolves — verify against current Google guidance/your publisher account.

Uninstall intent didn't remove the app

- **Root cause:** the **Uninstall** assignment isn't targeting the device's group, or the app is also

Required elsewhere (a conflicting Required assignment wins).

- **Fix:** ensure no other group assigns the app as **Required** to the same device; set **Uninstall** on

the correct group; sync. Verify with `ANDROIDAPP-Discovery.ps1`.

"Can't add a Google account" / enrollment-side failures

- **Root cause:** network reachability to Google/Microsoft endpoints blocked, or a **connector / AMAPI**

issue (out of scope for app delivery but blocks it upstream).

• **Fix:** confirm required endpoints are allowed (category global requirement 5). Connector/AMAPI-level problems are **high-impact** — handle under change control and **verify re-enrollment behavior against current Microsoft guidance / your tenant** before acting.

First moves for any app-delivery issue

1. `ANDROIDAPP-Scope.ps1` — is the binding healthy and app-sync recent/successful?
2. `ANDROIDAPP-Discovery.ps1` — is the app assigned, to the right group, with the right intent + track?
3. `ANDROIDAPP-Validation.ps1` — does the app have config + protection + a defined track (the four gates)?
4. Only then change configuration — preview with `-WhatIf`, keep the current set as your rollback.

7. Best Practices — Android Enterprise App Management via Managed Google Play (Intune)

Catalog governance

• **Approve deliberately, not broadly.** Every approved app is visible in the managed work Play Store — approve only what you deliver, identified by **exact package name + publisher**, and keep an owned list.

• **Set permission-change handling on purpose.** Choose how an app behaves when it requests **new permissions** on update (keep-approved vs re-review). Re-review is safer for sensitive apps; keep-approved avoids apps silently dropping out of the catalog — decide per app and record it.

• **Prefer store apps; use private LOB only when you must.** Private publishing adds a Google-side developer-account dependency and a slower change loop — reserve it for genuinely internal apps.

Assignment discipline

• **The `assign` action replaces ALL assignments.** Never hand-edit assignments blind — read the current set first (`ANDROIDAPP-Discovery.ps1` / `-WhatIf`), then write the **complete** intended set. This is the #1 way teams accidentally drop a group.

• **Match intent to enrollment model.** Required for fully managed / corporate-owned work profile / dedicated; Available for work-profile BYOD. Don't push Required to BYOD unless that is truly intended.

• **Assigned groups, not Dynamic, for app targeting** — predictable membership avoids apps appearing on the wrong devices as dynamic rules churn.

Managed configuration

• **Pre-configure everything you can.** Push accounts, server URLs, and feature flags via app-config policies so users get a zero-touch first run — fewer setup tickets, consistent config.

• **Pick the right profile applicability.** Author separate app-config policies for personally-owned work profile vs fully managed/dedicated/corporate-owned — the available keys and behavior differ.

• **OEMConfig is the sanctioned OEM path.** Drive Samsung Knox / Zebra settings through their OEMConfig app's schema rather than ad-hoc device settings; treat the schema/keys as vendor-authoritative.

App protection (APP / MAM)

• **Layer APP on top of device policy** for managed devices, and use **MAM-only** APP as the primary control on unenrolled / work-profile BYOD where you don't manage the whole device.

• **Standardize a baseline APP policy** (PIN, encryption, block copy/paste + save-as to personal, conditional launch on min OS/patch) and reuse it, rather than bespoke policies per app.

Update-track control

• **Pilot updates on a Beta / closed track** to a small group before the fleet moves. Keep Production on the stable track. Set **auto-update mode** intentionally (e.g. high priority for security-critical apps).

• **Don't let tracks drift.** Re-run `ANDROIDAPP-Discovery.ps1` periodically to confirm each app is on the intended track and mode.

Change safety

• **Pilot-first, always** — prove the four gates on a device per model before expanding.

• **High-impact = separate change window.** Connector re-binding, enrollment-model changes, and the **AMAPI** migration can affect existing devices and may involve **re-enrollment** — **verify against current Microsoft guidance / your tenant**; never bundle them with routine app changes.

• **Keep a rollback ready** — the current assignment set (from `-WhatIf`) is your restore point; unassign / revert config / revert track to recover.

• **Least privilege** — do routine app work as **Intune Administrator**; reserve Global Admin for the initial managed Google Play binding.

Hygiene

• **No secrets in automation.** Scripts authenticate interactively via `Connect-MgGraph` with least scopes — never embed credentials, tokens, or app secrets.

• **Document per app:** type, package name, intent per group, managed config keys, APP policy, and track.

8. Deliverables Summary — Android Enterprise App Management via Managed Google Play (Intune)

What you receive

A complete, self-guided package to deliver and govern Android apps through **managed Google Play** in Microsoft Intune — store, private (LOB), and web apps, managed configuration, app protection, and update-track control, targeted by enrollment model.

Documentation

• **Solution Overview** — app types on managed Google Play, the four levers (approve+assign, managed

config, app protection, update track), and how intent maps to each enrollment model.

• **Requirements Guide** — app-management-specific prerequisites (healthy connection, enrollment models,

app inventory + package names, private/web publishing, managed-config schema, APP prerequisites, assignment groups), each with Verify / Implement, on top of the six category global requirements.

• **Implementation Guide** — ordered steps: groups → baseline → approve store app → private/web →

assign to pilot (intent + track) → managed config → app protection → validate → expand, every object named.

• **Testing & Validation** — the four gates (Delivery, Configuration, Protection, Update track) as a

per-model pass/fail matrix, with automated checks and evidence to capture.

• **Troubleshooting** — symptom → root-cause → fix for the common failures (approved app never appears,

app not offered, config not applied, APP not enforced, wrong track, uninstall not taking).

• **Best Practices** — catalog governance, assignment discipline (the `assign`-replaces-all gotcha),

managed config, APP layering, track piloting, and change safety.

PowerShell tools (Microsoft Graph) — safe by default

Script	Purpose	Default mode
`ANDROIDAPP-Discovery.ps1`	Inventory approved managed Google Play apps + assignment posture (intent, group, track, auto-update mode)	Read-only

Script	Purpose	Default mode
`ANDROIDAPP-AssignConfig.ps1`	Stage an app **assignment** (intent + track + auto-update) and report managed-config attachment; write only with `-Commit`	`-WhatIf`
`ANDROIDAPP-AppProtection.ps1`	List Android app protection policies + assignments; stage/create a baseline APP policy with `-Commit`	Read / `-WhatIf`
`ANDROIDAPP-Validation.ps1`	Per-app report: has assignment? managed config? app protection? defined track/mode? — flags gaps	Read-only
`ANDROIDAPP-Scope.ps1`	Managed Google Play binding + app-sync health and scope check (pre-flight)	Read-only

All scripts use `Connect-MgGraph` with **least scopes**, contain **no credentials or secrets**, and gate every write behind `-Commit` (with `-WhatIf` preview).

Outcomes

- Apps delivered through managed Google Play with the **right type and intent** per enrollment model.
- Apps arrive **pre-configured** via managed app configuration (including OEMConfig at the pattern level).
- Corporate app data **protected** via Android app protection policies.
- Updates **controlled** by track and auto-update mode, piloted before the fleet.
- A **pilot-first**, gated, reversible rollout with a documented rollback for every change.

Assumptions & boundaries

- A **healthy managed Google Play binding** already exists (established by the connection-setup solution).
- Private LOB / web-app **publishing** happens on the **Google side** first; Intune approves and assigns.
- OEM (Samsung Knox / Zebra) and third-party app specifics are at the **pattern level** — vendor docs are

authoritative for schemas/keys.

- **AMAPI transition, web/account-driven enrollment, and re-enrollment behavior are evolving** — verify

against current Microsoft guidance / your tenant before high-impact changes.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

ANDROIDAPP-AppProtection.ps1

Purpose: Reports Android managed app protection policies and their app/group assignments (read mode). With -Commit it creates a conservative BASELINE Android APP policy (PIN required, encrypt org data, block copy/paste + save-as to personal locations) that you then assign in Intune — a safe starting point to harden. HOW TO RUN Report: powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-AppProtection.ps1 Preview: ...same... -Name "ANDROIDAPP Baseline" -WhatIf Create: ...same... -Name "ANDROIDAPP Baseline" -Commit Requires the Microsoft Graph SDK; read uses DeviceManagementApps.Read.All, -Commit needs DeviceManagementApps.ReadWrite.All. EXPECTED RESULTS A list of existing Android APP policies + assignments; with -Commit, a new unassigned baseline policy exists. PASS = the target apps have an APP policy enforcing your data controls. .NOTES White-label — Android Solutions. Read/-WhatIf by default; creates only with -Commit. Assign in Intune after review. No credentials stored. .EXAMPLE .\ANDROIDAPP-AppProtection.ps1 -WhatIf > [CmdletBinding()] param([string]$Name = 'ANDROIDAPP Baseline (review before assigning)', [switch]$Commit, [switch]$WhatIf)

How to run: Report: powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-AppProtection.ps1 Preview: ...same... -Name "ANDROIDAPP Baseline" -WhatIf Create: ...same... -Name "ANDROIDAPP Baseline" -Commit Requires the Microsoft Graph SDK; read uses DeviceManagementApps.Read.All, -Commit needs DeviceManagementApps.ReadWrite.All. EXPECTED RESULTS A list of existing Android APP policies + assignments; with -Commit, a new unassigned baseline policy exists. PASS = the target apps have an APP policy enforcing your data controls. .NOTES White-label — Android Solutions. Read/-WhatIf by default; creates only with -Commit. Assign in Intune after review. No credentials stored. .EXAMPLE .\ANDROIDAPP-AppProtection.ps1 -WhatIf > [CmdletBinding()] param([string]$Name = 'ANDROIDAPP Baseline (review before assigning)', [switch]$Commit, [switch]$WhatIf)

ANDROIDAPP-AssignConfig.ps1

Purpose: Previews (and, with -Commit, applies) an assignment for one managed Google Play app: intent, target group, update track(s), and auto-update mode. Because the Graph 'assign' action REPLACES ALL of an app's assignments, this reads the current set and writes the current set PLUS the new assignment — never clobbering existing groups. Also reports whether an app-config policy is attached for the app's package. HOW TO RUN Preview: powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-AssignConfig.ps1 -PackageId com.microsoft.office.outlook -GroupId <entra-group-guid> -Intent required -AutoUpdateMode default -WhatIf Apply: ...same... -Commit Requires the Microsoft Graph SDK; read uses DeviceManagementApps.Read.All, -Commit needs DeviceManagementApps.ReadWrite.All. EXPECTED RESULTS Prints current assignments + the intended new assignment (and app-config status). With -Commit, the app has the new assignment. PASS = the app shows the intended intent/group/track afterward. .NOTES White-label — Android Solutions. Safe by default (-WhatIf); writes only with -Commit. No credentials stored. .EXAMPLE .\ANDROIDAPP-AssignConfig.ps1 -PackageId com.microsoft.office.outlook -GroupId 1111-2222 -Intent required -WhatIf > [CmdletBinding()] param([Parameter(Mandatory)][string]$PackageId, [Parameter(Mandatory)][string]$GroupId, [ValidateSet('required','available','availableWithoutEnrollment','uninstall')][string]$Intent = 'required',

[ValidateSet('default','postponed','priority','unavailable')][string]$AutoUpdateMode = 'default', [string[]]$TrackIds, [switch]$Commit, [switch]$WhatIf)

How to run: Preview: powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-AssignConfig.ps1 -PackageId com.microsoft.office.outlook -GroupId <entra-group-guid> -Intent required -AutoUpdateMode default -WhatIf Apply: ...same... -Commit Requires the Microsoft Graph SDK; read uses DeviceManagementApps.Read.All, -Commit needs DeviceManagementApps.ReadWrite.All. EXPECTED RESULTS Prints current assignments + the intended new assignment (and app-config status). With -Commit, the app has the new assignment. PASS = the app shows the intended intent/group/track afterward. .NOTES White-label — Android Solutions. Safe by default (-WhatIf); writes only with -Commit. No credentials stored. .EXAMPLE .\ANDROIDAPP-AssignConfig.ps1 -PackageId com.microsoft.office.outlook -GroupId 1111-2222 -Intent required -WhatIf > [CmdletBinding()] param([Parameter(Mandatory)][string]$PackageId, [Parameter(Mandatory)][string]$GroupId, [ValidateSet('required','available','availableWithoutEnrollment','uninstall')][string]$Intent = 'required', [ValidateSet('default','postponed','priority','unavailable')][string]$AutoUpdateMode = 'default', [string[]]$TrackIds, [switch]$Commit, [switch]$WhatIf)

ANDROIDAPP-Discovery.ps1

Purpose: Lists Android Enterprise (managed Google Play) apps published to Intune — store, private, and web — with each app's assignments: intent, target group, update track(s), and auto-update mode. Read-only; safe to run any time to baseline your app-delivery posture. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-Discovery.ps1 -CsvPath C:\Temp\android-apps.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS One line per app+assignment (Intent / GroupId / Tracks / AutoUpdate), plus an optional CSV. PASS = every intended app appears with the expected intent, group, and track. .NOTES White-label — Android Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ANDROIDAPP-Discovery.ps1 -WhatIf > [CmdletBinding()] param([string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-Discovery.ps1 -CsvPath C:\Temp\android-apps.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS One line per app+assignment (Intent / GroupId / Tracks / AutoUpdate), plus an optional CSV. PASS = every intended app appears with the expected intent, group, and track. .NOTES White-label — Android Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ANDROIDAPP-Discovery.ps1 -WhatIf > [CmdletBinding()] param([string]$CsvPath)

ANDROIDAPP-Scope.ps1

Purpose: Pre-flight check before app work: reports the managed Google Play (Android managed store account enterprise) binding — bind status, owner, enrollment target, and the LAST APP SYNC status/time — plus a count of managed Google Play apps and app-config policies currently in the tenant. A stale or errored sync is the top reason an approved app never reaches devices. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-Scope.ps1 Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementApps.Read.All. EXPECTED RESULTS Binding status + last app-sync time, and app/config counts. PASS = bound, sync recent/successful. .NOTES White-label — Android Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ANDROIDAPP-Scope.ps1 -WhatIf > [CmdletBinding()] param()

How to run: powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-Scope.ps1 Requires the Microsoft Graph SDK and DeviceManagementConfiguration.Read.All + DeviceManagementApps.Read.All.

EXPECTED RESULTS Binding status + last app-sync time, and app/config counts. PASS = bound, sync recent/successful. .NOTES White-label — Android Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ANDROIDAPP-Scope.ps1 -WhatIf > [CmdletBinding()] param()

ANDROIDAPP-Validation.ps1

Purpose: For each managed Google Play app, reports whether it has (1) an assignment, (2) a managed app configuration for its package, (3) an Android app protection policy referencing it, and (4) a defined update track / auto-update mode — the four validation gates. Flags gaps so you know an app is fully governed before you expand. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-Validation.ps1 -CsvPath C:\Temp\android-validate.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A table: App / Assigned / HasConfig / HasAPP / Track-or-AutoUpdate, with a gap count. PASS = intended apps are Assigned + Configured (+ Protected where required) + track defined. .NOTES White-label — Android Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ANDROIDAPP-Validation.ps1 -WhatIf > [CmdletBinding()] param([string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\ANDROIDAPP-Validation.ps1 -CsvPath C:\Temp\android-validate.csv Requires the Microsoft Graph SDK and DeviceManagementApps.Read.All. EXPECTED RESULTS A table: App / Assigned / HasConfig / HasAPP / Track-or-AutoUpdate, with a gap count. PASS = intended apps are Assigned + Configured (+ Protected where required) + track defined. .NOTES White-label — Android Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ANDROIDAPP-Validation.ps1 -WhatIf > [CmdletBinding()] param([string]$CsvPath)